Projektbeskrivning

Catan

2025-03-12

Projektmedlemmar:
Jimi Henricsson, jimhe280@student.liu.se
Viggo Scheding Jauhola, vigsc133@student.liu.se

Handledare:
Simon Hansson <handledare@liu.se>

Innehåll

1. Introduktion till projektet 2
2. Ytterligare bakgrundsinformation 2
3. Milstolpar 2
4. Övriga implementationsförberedelser 4
5. Utveckling och samarbete 4
6. Implementationsbeskrivning 6
   6.1. Milstolpar 6
   6.2. Dokumentation för programstruktur, med UML-diagram 6
7. Användarmanual 7

Projektplan

1. Introduktion till projektet

Vi har valt att skapa Catan som ett datorprogram. Catan är ett brädspel som går ut på att bygga ut och utveckla byar och städer för att på så sätt samla in råvaror för att kunna bygga mer och då få vinstpoäng.



2. Ytterligare bakgrundsinformation

Följande ska finnas med i spelet:

- 19 landbrickor (4 vete, ull och trä, 3 tegel och sten, 1 öken)
- Havsram med hamnar
- 18 siffermarkörer (1st 2a och 12a, 2st 3-11)
- råvarukort av varje resurs
- 25 utvecklingskort (14 riddarkort, 2 vägbygge-kort, 2 skörd-kort, 2 monopol-kort, 5 vinstpoäng-kort)
- 2 specialkort (Längsta handelsväg, största riddarmakt)
- 96 spelpjäser (24 av varje färg (4 städer, 5 byar, 15 vägar))
- 1 rövarfigur
- 2 tärningar

Spelet börjar med att första spelaren väljer ett ställe att placera en by på och sedan en väg ut från den byn åt valfritt håll. Det går sedan medurs där nästa spelare får göra samma sak tills sista spelaren. Sedan går det likadant fast varvet tillbaka så att sista spelaren lägger båda sina byar och vägar samtidigt och första spelaren lägger sin andra by och väg sist. När alla spelare har satt ut sina två byar får varje spelare välja en av byarna att få startresurser från (resurserna som finns runtom byn). För att bygga en ny by måste man ha en väg till den och den får inte byggas intill en annan by. Vägar måste vara intill en by eller annan väg man äger för att kunna byggas. Städer byggs genom att uppgradera en by, en by måste alltså finnas innan man kan göra en stad.

Kostnader för att bygga saker är följande:

- 1 tegel och 1 trä för väg
- 1 tegel, 1 trä, 1 ull och 1 vete för en by
- 3 vete och 2 sten för stad
- 1 vete, 1 sten, 1 ull för utvecklingskort

En runda börjar med att man slår tärningarna och resurser delas ut till varje spelare som har en by eller stad intill en ruta som har talet som slogs, 1 av resursen om det är en by, 2 om det är en stad. Sedan får aktiv spelare göra valfritt många köp och handlingar som denne kan och vill göra. Rundan är sedan över när aktiva spelaren känner sig klar och går då vidare till nästa spelare. Om man slår en sjua måste alla spelare som har över 7 kort vaska hälften av sina kort avrundat nedåt. Man kan närsomhelst under sin tur byta 4 av en resurs mot 1 valfri med banken. Att ha en by/stad liggande intill en hamn kan låsa upp annan byteshandel också. Hamnarna kan göra så att man istället kan byta 3:1 eller byta 2:1 av specifik resurs.

Byar ger 1 poäng, städer 2 poäng, specialkort (längsta handelsväg och största riddarmakt) ger 2 poäng vardera, vinstpoäng-kort från utvecklingskorten ger 1 poäng vardera. Man vinner direkt när man har 10 poäng.

Längsta handelsväg-kortet får man direkt när man har byggt en sammanhängande väg av 5 vägar. Kortet behålls då tills någon annan spelare bygger en längre sammanhängande väg då kortet direkt överförs till den nya spelaren med längst väg.

Största riddarmakt-kortet får man direkt när man har spelat sitt tredje riddar-kort från utvecklingskorten. Detta behålls på samma sätt som längsta handelsväg-kortet tills någon annan spelare har spelat ett mer riddarkort än vad du har, då kortet direkt överförs.

För fler regler se regelboken för fan.

Regelbok:

https://www.catan.com/sites/default/files/2021-06/catan_base_rules_2020_200707.pdf

3. Milstolpar

#	Beskrivning
1	Det finns datatyper för olika hexagoner samt en grafisk komponent som kan visa en hexagon med färg beroende på datatyp. En testklass kan skapa en slumpmässig hexagon och visa den grafiskt i ett fönster. Det går att stänga fönstret genom att klicka på stängknappen.
2	Ändra grafiska komponenten så att den kan visa 19 mindre hexagoner som innerdelen av en spelplan ska se ut. Dessa ska också vara färgkodade beroende på datatyp. Testklassen kan slumpa fram hexagonerna och visa de grafiskt i ett fönster.
3	Lägg till så att rätt antal av varje hexagon-typ slumpas ut på planen.

4	Utöka grafiska komponenten så att den kan visa en cirkel med ett tal i mitten av varje hexagon. Testklassen kan slumpa tal mellan 2-12 i dessa. På något sätt se till att det finns en koppling mellan hexagon och tillhörande tal.
5	Lägg till så att rätt antal av varje tal slumpas ut och att öken-rutan inte får något tal.
6	Skapa datatyper för by, stad. Skapa lista (eller motsvarande) med möjliga placeringspositioner samt vilken datatyp som finns på positionen (by, stad eller empty). Testklassen kan slumpa fram olika typer för de olika positionerna och skriva ut i terminalen.
7	Lägg till så att byarna och städerna ritas ut som cirklar i olika färger på rätt positioner på planen.
8	Skapa datatyp för väg. Skapa lista (eller motsvarande) med möjliga placeringspositioner samt vilken datatyp som finns på positionen (väg eller empty). Testklassen kan slumpa fram olika typer för de olika positionerna och rita ut grafiskt som cirklar med unik färg.
9	Lägger till ramen (vattnet runt spelplanen med hamnar) som bakgrundsbild till fönstret och anpassar komponenternas storlek och position för att allt ska passa in i ramen.
10	Införa knappar som tillåter spelaren att placera byar, städer och vägar enligt spelets regler. Tre knappar som väljer vad man vill placera, och som dessutom visar knappar på alla möjliga positioner man kan placera komponenten på.
11	Skapa datatyper för resurskort. Skapa spelarklass som innehåller information om resurskort som tillhör spelaren och antal poäng spelaren har. På något sätt skapa en anknytning mellan spelaren och de pjäser som spelaren äger för att räkna poängen och tilldela resurser till rätt spelare. Lägg till textruta för representation av poäng.
12	Kontrollera resurser för utplacering av byar, städer och vägar samt se till att resurserna tas bort när du bygger saker. Lägga till textruta för representation av resurser.
13	Implementera två slumpgeneratorer som slumpar fram tal mellan 1-6 för att simulera två tärningar. Skapa knapp för att slå tärningen. Lägger till osynliga hexagoner utanför spelbrädet som inte ger några resurser. Tilldela resurser till spelaren utifrån tärningskastets resultat och rutorna som by/stad står intill. Om någon spelare har fler än 7 kort när en sjua slås ska hälften av korten slumpas bort.
14	Implementera win condition vid 10 poäng och göra så att spelet avslutas och vinnaren skrivs ut i terminalen.
15	Implementera rövaren enligt reglerna och göra så att rövaren blockerar rutan den står på. Göra så att om man slår en sjua så får man placera om rövaren var man vill. Rita ut rövaren med en svart prick på spelbrädet.

16	Implementera hamnar med lista för position och villkor för byte samt se till att spelarklassen vet om spelaren har en by/stad intill en hamn.
17	Implementera utvecklingskort med olika effekter. Implementera knapp i gränssnittet för att köpa utvecklingskort samt på något sätt visa för spelaren vilket utvecklingskort som dragits.
18	Gör så att man kan spela utvecklingskorten och att de har önskad effekt.
19	Lägg till framstegskort för störst riddarmakt och längsta handelsväg.
20	Implementera flerspelarläge genom lokalt nätverk för 2 spelare. Gör så att tärning kan slås en gång per runda och rundor kan avslutas m.h.a knapp. Gör så att rundorna går mellan spelarna. Implementera början av spelet korrekt.
21	Lägg till grafiska representationer för resurskort och utvecklingskort. Ändra representationen av byar/städer och vägar samt rövaren till bilder. Ändra hexagonernas representation till bilder också. Grafiska representationer till allt annat som kan behövas.
22	Färdigställ byteshandel med banken och hamnar. Gör om så att när en sjua slås så väljer du hälften av korten som ska bort istället för att de slumpas bort.
23	Utveckla flerspelarläget till flera spelare.
…	

4. Övriga implementationsförberedelser

Vi tänker ha:

- En klass för brädet som innehåller mycket av informationen och reglerna för spelet.
- En spelarklass som innehåller information om spelarens ägodelar och poäng.
- En komponentklass för själva spelplanskomponenten som ska ritas upp.
- En visarklass som skapar panelen med spelet och ritar ut komponenten och annat som ska synas.
- Enum-klass för olika resurser.
- En hexagonklass som lagrar hexagonens grafiska representation, tillhörande resurs, tilldelat tal, samt om det står en rövare på hexagonen eller ej.
- Enum-klass för byar och städer.
- En byggnadsklass (för byar och städer) som innehåller grafiska representationen, om det är en by eller stad, vilken spelare som äger byggnaden och sedan en funktion för att kolla på intilliggande hexagoner och tilldela resurser till ägaren om hexagonen har rätt nummer.

- En klass för väg som innehåller grafiska representationen och vägens ägare. Möjligtvis också vägens position.

I brädesklassen tänker vi ha listor eller liknande för att ange positioner där byar och vägar kan placeras. En lista för byggnader som innehåller <position, byggnad> där byggnad är null om det inte finns någon. Likadant med vägar har vi en lista med <position, väg>.

5. Utveckling och samarbete

Vi jobbar på och försöker nå vår milstolpar i tid och vara i fas och siktar på högre betyg så länge det inte känns som alltför mycket jobb att få det. Vi jobbar tillsammans och delar även ibland upp arbetsuppgifter för att jobba lite enskilt ibland och vara effektiva.

6. Implementationsbeskrivning

6.1. Milstolpar

1-5: Helt genomfört.

6: Helt genomförd men med en lista för placeringspositioner och en annan lista för settlements.

7: Helt genomförd men by = cirkel, stad = kvadrat istället för olika färger.

8: Vi ändrade så vägar görs istället mellan två hörnpositioner och ritas ut som en linje mellan dessa. Vägarna sparas i en lista men vi har ingen lista för giltiga vägpositioner.

9-12: Helt genomfört.

13: Helt genomfört förutom osynliga hexagoner runt som vi slopade.

14-15: Helt genomfört.

16: Helt genomfört men harbors är objekt och sparas i en lista i board.

17: Helt genomfört

18: Påbörjat men inte helt klart

19-23: Ej klart

6.2. Dokumentation för programstruktur, med UML-diagram

Övergripande programstruktur

Vi har implementerat ett brädspel inspirerat av *Catan*, där spelet kontrolleras genom knapptryckningar. Den centrala klassen för användarinteraktion är *CatanViewer*, som hanterar input från användaren i form av knapptryckningar. Dessa knapptryckningar används för att manipulera spelets tillstånd och exempelvis bygga vägar eller byar, samt slå tärningen. De operationer som påverkar spelet utförs sedan genom att anropa metoder i klassen *Board*.

Board representerar spelets logik och innehåller information om brädet, resurser, spelare, samt vilka byggnader och vägar som existerar. När någon förändring sker i spelets tillstånd, till exempel att en spelare bygger en väg eller får en resurs, uppdateras detta i *Board*.

Efter att en förändring har skett kallas *repaint()* på komponenten *CatanComponent*, som ansvarar för att rita om spelplanen. Denna komponent itererar över de objekt som implementerar gränssnittet *VisibleGameObject*, till exempel *Hexagon*, *Road*, *Village* och *City*, och anropar deras *draw(Graphics2D g2d)*-metoder.

Flödet är alltså:

1. Användaren klickar på en knapp i *CatanViewer*.
2. *Board* uppdaterar spelets tillstånd utifrån åtgärden.
3. *Board* notifierar registrerade lyssnare via *BoardListener*.
4. *CatanComponent* kallar *repaint()* för att rita ut den nya spelstatusen.

Objektorienterad design

Programmet är uppbyggt enligt objektorienterade principer och visar tydlig användning av flera centrala OO-koncept:

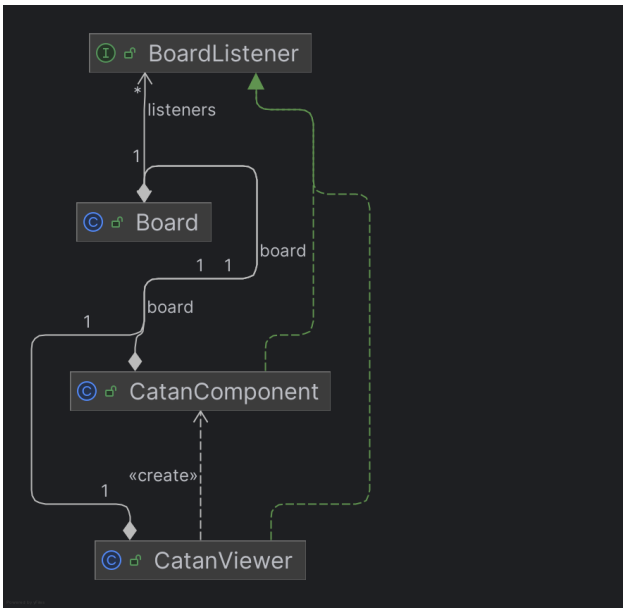
- **Inkapsling**: Alla interna fält i klasser som *Board*, *Player*, *Hexagon* och *ResourceCard* är privata, vilket säkerställer att deras data inte kan modifieras direkt utifrån. Åtkomst sker istället via publika metoder.
- **Interface och subtypspolymorfism**: I programmet finns gränssnittet *VisibleGameObject*. Detta interface används av alla objekt som ska kunna ritas på spelplanen. Det innehåller metoden *draw(Graphics2D g2d)*, vilket möjliggör polymorf rendering i *CatanComponent* och ser till så att alla synliga objekt måste kunna rita upp sig själva.

Klasser som implementerar *VisibleGameObject*:
- *Hexagon*
- *Road*
- *Settlement* (abstrakt klass)
  - *Village*
  - *City*

Genom detta interface kan *CatanComponent* behandla alla objekt likadant ur ett ritperspektiv, oavsett vilken klass det faktiskt rör sig om.
- **Abstrakta klasser och ärvning**: *Settlement* är en abstrakt klass som innehåller gemensam funktionalitet för både *Village* och *City*. Det möjliggör återanvändning av kod för till exempel positionering och spelartillhörighet, samtidigt som de två subklasserna kan specificera sina egna egenskaper och beteenden.
- **Overriding och specialisering**: Varje klass som implementerar *VisibleGameObject* overridar *draw*-metoden för att implementera sin specifika rendering. Till exempel ritar *Hexagon* en resursbricka med nummerbricka, medan *City* ritas som en uppgraderad version av *Village*.
- **Komposition**: Klasser som *Board* och *Player* använder sig av komposition för att hålla reda på spelrelaterade objekt. *Board* håller till exempel reda på resurser (*ResourceCard*), byggnader (*Settlement*, *Road*) och spelare (*Player*) och *Player* på harbors (*Harbor*).
- **Observer**: *Board* implementerar ett lyssnarsystem genom att hålla en lista av *BoardListener*-objekt. När något ändras i spelets tillstånd så anropas en metod på dessa lyssnare, vilket inkluderar *CatanComponent* och *CatanViewer*. Detta skapar en lös koppling mellan modell och vy och förenklar vidareutveckling.

UML-diagram och beskrivningar

Spelmodell och input

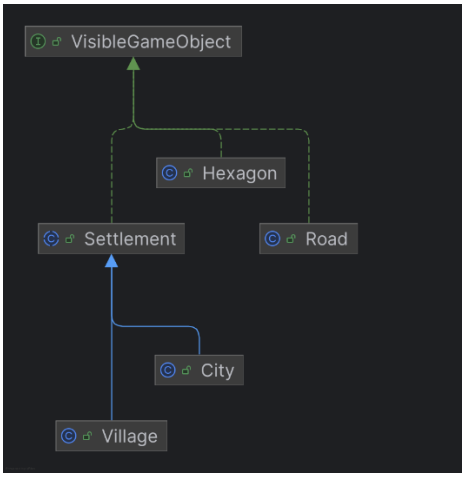


Klasser:
- *CatanViewer*

- *Board*
- *BoardListener* (interface)
- *CatanComponent*

Detta diagram visar huvudflödet mellan användarinput och modelluppdatering. *CatanViewer* agerar som användargränssnitt och förmedlar knapptryckningar till *Board*, som i sin tur uppdaterar speldata och informerar sina lyssnare med hjälp av *BoardListener*. *CatanComponent* är en lyssnare som svarar på dessa förändringar genom att rita om komponenten.

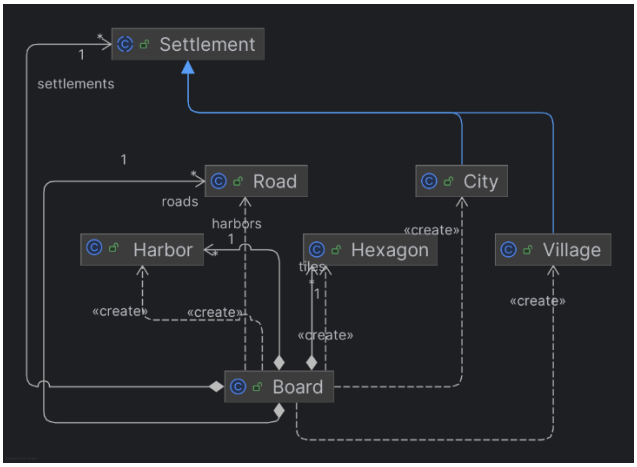
Visuella spelobjekt



Klasser:
- *VisibleGameObject* (interface)
  - *Hexagon*
  - *Road*
  - *Settlement* (abstrakt klass)
    - *Village*
    - *City*

Detta diagram visar sambandet mellan det grafiska interfacet *VisibleGameObject* och dess implementationer. *CatanComponent* itererar över en samling av dessa objekt och anropar *draw()* polymorft. Ärvningskedjan från *Settlement* till *Village* och *City* visar på återanvändning av kod samt möjligheten att särskilja beteenden i underklasser.

Resurser och byggnader



Klasser:
- *Board*
- *Harbor*
- *Road*
- *Hexagon*

- *Settlement* (+ subklasser)

Diagrammet illustrerar hur spelobjekt och byggnader hanteras. *Board* innehåller listor av *Road*, *Village*, *City*, *Harbor* och *Hexagon*. *Board* skapar även objekt av dessa typer. Detta diagram ger en tydlig översikt över spelplanens objekt och hur dessa lagras och skapas.

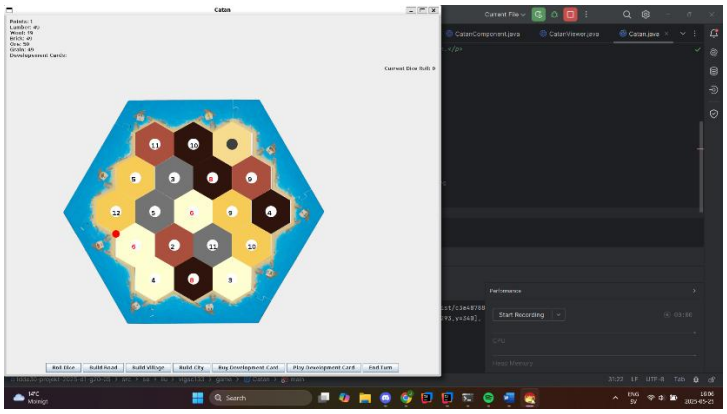
Slutsatser och förslag till vidareutveckling

Kodstrukturen är välorganiserad med tydlig separation mellan användargränssnitt, spelmodell och rendering. Den objektorienterade designen med gränssnitt och ärvning gör systemet lätt att bygga ut. Exempelvis skulle man enkelt kunna införa nya typer av spelobjekt (t.ex. specialbyggnader) genom att låta dem implementera *VisibleGameObject*.

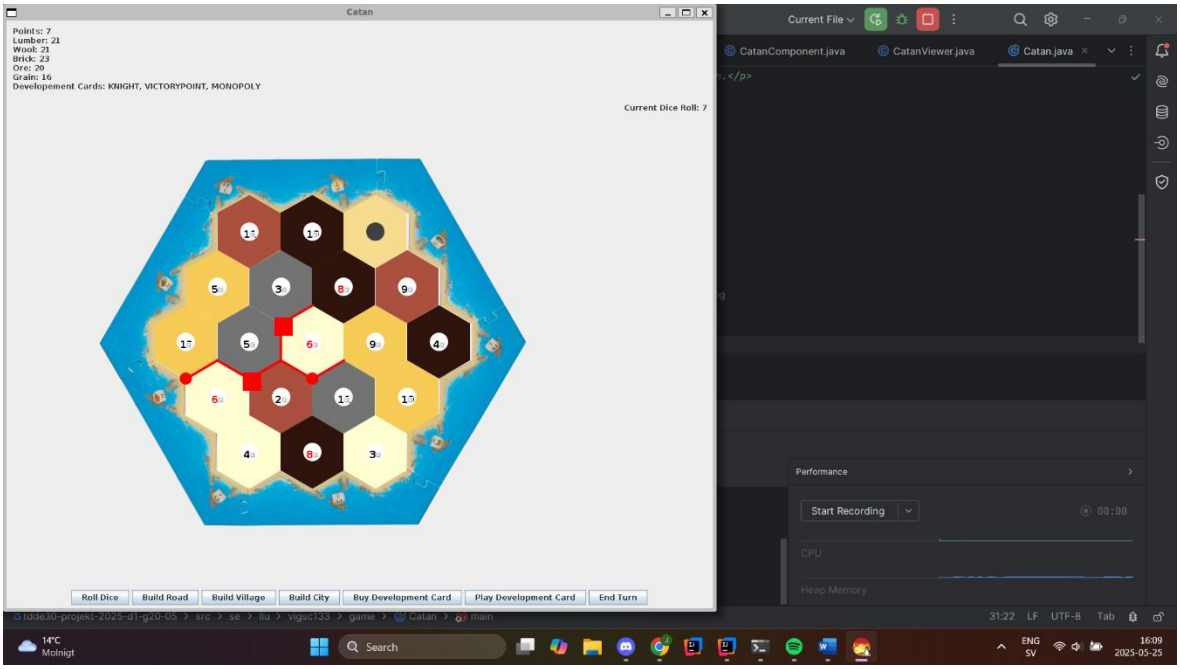
För vidareutveckling hade det varit användbart att ha ännu striktare separation mellan modell och vy. Det hade kanske varit möjligt att lägga till en controller-komponent mellan *CatanViewer* och *Board* för att renodla ansvar ytterligare och ha ansvaret kring knapptryckningar.

7. Användarmanual

För att starta programmet kör man main-metoden i main-klassen *Catan*. En slumpad (enligt reglerna) spelplan dyker då upp med en by som man börjar med (istället för att välja och placera vilket vi inte hann implementera).



Spelet styrs sedan genom knapptryckningar av knapparna på nedre delen av fönstret för att utföra olika handlingar. *Roll dice* slår tärningarna och delar ut resurser och liknande enligt reglerna. Om man slår en sjua får man upp knappar för att välja vart man vill flytta rövaren (som i reglerna).

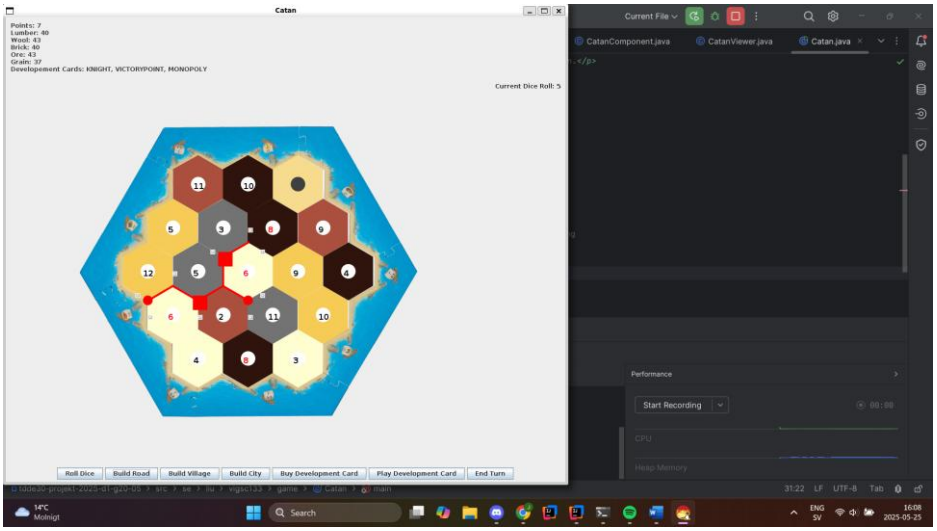


End turn avslutar din runda, med det menas att du kan slå tärningarna igen och om du har 10 eller mer poäng avslutas spelet och du vinner.

```
/home/jimih/.jdks/ms-21.0.6/bin/java
Spelet slut! Vinnare: Player1

Process finished with exit code 0
```

De andra knapparna är till för att bygga eller köpa nya byggnader och kort som vägar och byar. Dessa knappar fungerar så att man klickar på dem, sedan dyker nya knappar eller dialogrutor upp om man har råd och kan bygga eller köpa det man försöker bygga/köpa. Om man försöker bygga/köpa något man inte har råd med eller inte har möjlighet att placera ut händer ingenting. Om man har råd och möjlighet att placera ut det man försöker bygga dyker nya knappar upp på alla möjliga placeringspositioner, där det sedan byggs på platsen man väljer.



I övre hörnen av fönstret kan man också se all spelinformation som till exempel senaste tärningskastet, ägda resurser och poäng.